Luleå Technical University
School of Computer Science, Electrical and Space Engineering
Luleå, Sweden

Final Report, Architecture Document
Group 14
D7065E, Embedded Intelligence at the Edge

FIRE DETECTION AND AUTONOMOUS RESPONSE

Ekpenyong Esu Ekpenyong
ekpekp-5@student.ltu.se

Examiner: Johan Kristiansson
LTU, Luleå

12th October 2026

Contents

1 Summary **1**

2 Use case and building context (BuildSim) **1**

3 Requirements **3**

4 Architecture **4**
4.1 Context diagram (C4 Level 1) 4
4.2 Container diagram (C4 Level 2) 5
4.3 Component diagram (C4 Level 3): inside the firelab service 6
4.4 Design decisions and justifications 8

5 Interfaces and communication **9**

6 Simulating the sensor values (the physical model) **11**

7 Autonomous services and data engineering **12**
7.1 The autonomous service 12
7.2 The data pipeline 15

8 Behaviour **15**
8.1 Key scenario, dynamic diagram 15
8.2 Process lifecycle, state machine 16

9 Deployment and component placement **17**

10 Test plan **18**

11 Evaluation and experiments **19**

12 Dashboard **24**

13 Risks and critical reflection **26**

1. Summary

This system watches an office building for fire. It acts on what it finds without waiting for a person. Emulated smoke, carbon monoxide and temperature detectors measure the true state of each room, but not perfectly. A detector turns their readings into a probability of fire for each room. There are two detectors to choose from. One is a hand-written fusion rule: a fixed formula that combines the sensor readings into one score. The other is a language model that the system asks over the network. Each room has its own state machine. It climbs from NORMAL through INVESTIGATING and PRE-ALARM to CONFIRMED. When a room is confirmed, the system releases the sprinkler and closes the fire door. It also sends everyone out of the building along routes that avoid the alarming rooms. The building, its rooms and its 3D view come from BuildSim. This project adds the fire physics, the sensors, the intelligence and the response around it.

One principle guides every decision below: **nothing that comes after the sensors is allowed to see the truth**. The physics knows the real temperature of every room. The detector, the agent and the safety rules (interlocks) only see what the sensors reported. The whole project studies the gap between those two views. The dashboard shows the two side by side. So a false alarm or a missed fire shows up as a mismatch on screen, not hidden in a log.

Field	Value
Project title	Fire Detection and Autonomous Response
Team	Group 14, Ekpenyong Esu Ekpenyong
Use case	Fire detection from several sensor types, a checked alarm, sprinklers and fire doors under safety rules, and escape routes in a three-storey building
Repository	`https://github.com/Ekpenyong-Esu/BuildSim-FireDetection`
Version / date	v10.1 / 2026-10-11

Changes since the proposal. Three things changed. Each one changed because building the system taught me something the proposal did not know.

- **One process instead of a dozen.** The proposal sketched nine services joined by MQTT, with ThingsBoard for storage and dashboards. Its own risk list said the hardest part would be keeping a dozen processes in step in simulated time. That risk turned out to be the whole problem. At forty times real time, a reading sent over a broker is already out of date when the subscriber sees it. The physics, the sensors and the agent must agree on what "now" is. Otherwise a waiting time means nothing. So I built the system as one tick loop with four zones inside it. The proposal wanted the parts kept apart by process boundaries. I keep them apart with import rules instead, and a test checks those rules (Section 4.2). Section 13 discusses the trade-off.
- **The second detector is a language model, not an isolation forest.** The proposal planned to compare a hand-written fusion rule with a trained anomaly detector. The rule alone caught every real fire. It raised no false alarm at any noise level I tested (Section 11). The only data to train a forest on would have been the simulator's own output. So I made the promised comparison with a different second opinion: a language model behind an OpenAI-compatible endpoint. The model gets the same readings the rule sees and nothing else. I ask it for a probability, its evidence and a reason. It follows the same two-method protocol, so the agent, the safety rules and the dashboard did not change. Section 7.1 says how I keep it off the tick. Section 11 reports the comparison. In short, the model catches what the rule catches, but later and much more slowly. Its answers are also not repeatable. The rule stays the default.
- **Holding smoke back versus getting people out became a safety rule, not a policy.** The proposal expected to tune a trade-off. In practice the safe rule is simple, and it should not be tunable. A fire door may not close on a room that is occupied or that lies on someone's escape route. Doors never lock. The agent asks and the safety rule decides. The request stands until it is safe to carry out.

2. Use case and building context (BuildSim)

The building. The system runs on the A-building at Luleå University of Technology, as BuildSim models it. It has three storeys, 956 named spaces, 1005 doorways and 32 stairwells. It has seven real exits, all on the

ground floor. BuildSim owns the floor plans and the walkable graph. This graph joins rooms through their doors, and joins the storeys through the stairwells. BuildSim also owns an equipment tree, where devices are registered, and a 3D viewer fed over WebSocket. I did not modify BuildSim. I only added sensors to it and drew on it.

What BuildSim does not provide is anything that changes over time. There is no fire, no smoke, no people and no sensor physics. This project adds all of that. firelab loads the floor plans once. It turns them into a graph of rooms linked through their doors, where an open or closed door changes the link. From then on, it writes back to BuildSim about once per simulated second. It writes floor heat maps of the true temperature, smoke and CO. It writes particle effects for fire, smoke and running sprinklers. It also writes the agent's alarm markers, the fire-door state, every occupant's position, and each device's latest reading into the equipment tree. Two things go into the viewer session instead of the building, because BuildSim keeps them per browser tab. These are the coloured outlines of alarming rooms and the escape route.

Actors. The **operator** watches the control panel. The operator chooses what to set on fire, injects sensor faults and picks which detector scores the rooms. The operator can also switch the system into supervised mode. In that mode the agent still decides, but a person has to press the button. The **occupants** are simulated people. They affect nothing until the alarm sends them walking. If someone is in a room, that room's door cannot be closed. Neither actor is on the control path. The loop from reading to sprinkler runs on its own.

The scenario the demo tells. A fast flaming fire starts in office A3505 on the top floor. Fifty-one seconds later, the smoke reading has risen enough for the room to start INVESTIGATING. At 81 s it is in PRE-ALARM. At 106 s the room CONFIRMS. By then the probability of fire has stayed above 0.75 for the full confirmation waiting time. Three things happen in the same tick. The sprinkler is released, because the temperature reading backs up the smoke. The fire door closes, because the room is empty and on nobody's route. All 130 people in the building start walking to a ground-floor exit, on routes that avoid the alarming rooms. The last person is out 111 s after the alarm. By contrast, the kitchen-cooking preset in the same room makes smoke and warmth for fifteen minutes. It never gets past INVESTIGATING, because its CO never rises.

Why it matters. A smoke detector cannot tell a fire from burnt toast, a dusty corridor or its own drifting element. Real buildings are evacuated for all three. As a result, people learn to ignore alarms. Telling these apart takes several sensor types, combined over time and across neighbouring rooms. That is a real job for an autonomous service. The response has a real conflict inside it too. Closing a fire door holds the smoke back, but it also takes away an escape route. Three things drive the requirements that follow: how fast the system reacts, how well it copes with broken sensors, and how safe its actions are.

Where it sits. Fire detection is one of several systems that could run on the same building. The others in the course are ventilation, security and energy management. Each one adds sensors to BuildSim through the same API. The same facilities staff would watch all of them. Figure 1 shows the landscape. This document designs the highlighted system. One part of it reaches beyond the laptop. When the language model is the chosen detector, the service asks a chat endpoint over HTTPS about one room at a time. This is optional. Without it, or when it is slow, the rule scores every room.

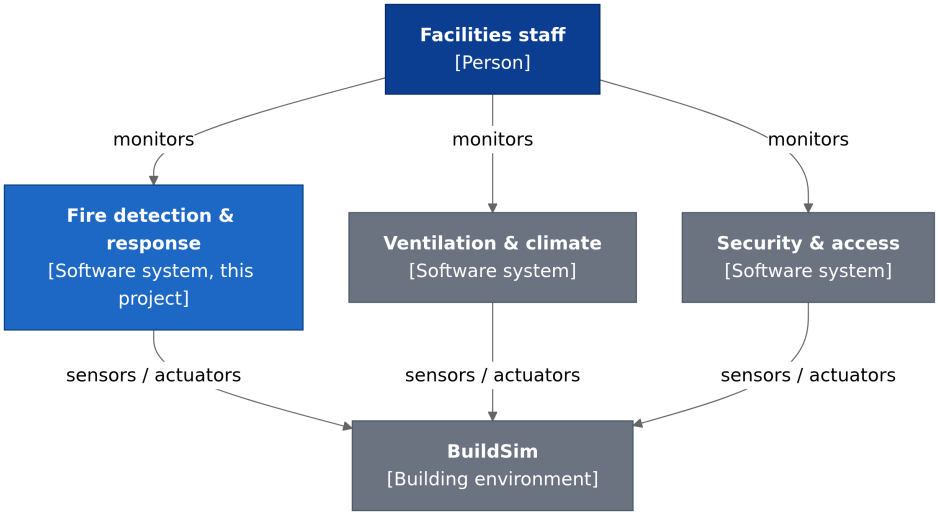


Figure 1: *C4 System Landscape. Several building systems share one BuildSim. This report designs the highlighted one.*

3. Requirements

Every requirement is a testable statement with a fixed ID. Functional requirements (FR) say what the system does. Safety requirements (SR) say what it must never do, whatever the agent decides. Non-functional requirements (NFR) say how well it must work. The last column names the test or experiment that checks each one. The test plan in Section 10 and the results in Section 11 close the loop.

ID	Requirement	Priority	Acceptance criterion	Verified by
FR-01	Confirm a flaming fire from sensor readings alone	Must	Room reaches CONFIRMED within 180 s of ignition	`test_agent`, `test_detector`, E1
FR-02	Confirm a smouldering fire, which has almost no heat	Must	CONFIRMED within 300 s (the scoreboard's miss limit)	E1
FR-03	Raise no alarm for cooking, dust, steam, or a drifting sensor	Must	Zero false alarms across all false-alarm presets at sensor noise 0–3×	`test_scoring`, E1, E3
FR-04	On confirmation: release sprinkler, close the fire door, evacuate	Must	All three commands are issued in the confirming tick and logged in the journal with a reason	`test_agent`, E1
FR-05	Route every evacuee to a real ground-floor exit avoiding alarming rooms	Must	No route crosses an alarming room while a clear one exists. Upper floors go down the stairs	`test_evacuation`, `test_walkways`, `test_multifloor`
FR-06	Explain every decision	Must	P(fire), the probability of fire, is the sum of named contributions. Or it is the model's answer with its evidence and a one-line reason. Every command and every refusal carries a reason	`test_detector`, `test_llm_detector`, dashboard
FR-07	Score the run against ground truth	Should	Detections, misses, false alarms, alarm delay and RSET (the time everyone needs to get out) are reported live and can be exported as CSV	`test_scoring`, `test_history`
FR-08	A second detector behind the same protocol, switchable while running	Should	The language model scores rooms from readings only. The agent, safety rules and panel are unchanged. The panel can switch back	`test_llm_detector`, `test_layering`, E7
SR-01	Water needs a heat reading to back it up	Must	A sprinkler request is refused while the room's true temperature is below 35 °C	`test_interlocks`, E1

ID	Requirement	Priority	Acceptance criterion	Verified by
SR-02	Fire doors never lock	Must	A lock command is refused unconditionally	`test_interlocks`
SR-03	Never seal people in	Must	A close command is refused while the room is occupied or on an active escape route	`test_interlocks`, `test_publisher_doors`
NFR-01	Keep up with the clock	Must	One engine tick over the full building finishes inside the 250 ms tick budget at 1×. I measure the breaking point at 40×	E5
NFR-02	Survive a lying sensor	Must	A stuck, dropping-out or drifting smoke detector does not stop a flaming fire being confirmed	`test_sensing`, E4
NFR-03	Be repeatable	Must	The same seed gives the same run, tick for tick	`test_reset`, E1 (rerun)
NFR-04	Keep the truth out of the intelligence	Must	No module downstream of the sensors imports or reads the true room state	`test_layering`
NFR-05	Not flood BuildSim	Should	At most 24 sensor writes per publish. The viewer route and highlights are sent only when they change	`test_publishing_route`
NFR-06	Never wait for the model	Must	A tick never waits on the network. The rule scores any room with no fresh answer. A badly formed answer is refused, not guessed at	`test_llm_detector`
NFR-07	Never outrun the model	Must	An answer older than 60 simulated seconds is not used. The speed dial is capped at what measured answer times allow, and the journal says so	`test_pacing`, `test_llm_detector`, E7

There is a tension in this table. FR-01 and FR-02 want the alarm early. FR-03 wants it late enough to be sure. SR-01 to SR-03 may refuse the response the alarm asks for. The waiting times in the state machine settle the first tension. The safety rules settle the second. The evaluation reports where each of these compromises costs something. NFR-06 and NFR-07 also pull against each other. A detector that answers eight seconds late is useless at forty times real time. So either the clock slows down or the rule takes over. The system does the first and falls back to the second. The safety requirements apply whichever detector is installed. That is why I keep the safety rules outside the intelligence.

Traceability. The five architecture viewpoints map onto this document. They are context (Section 4.1), functional (Sections 4.2 and 4.3), information (Sections 5 to 7), behavioural (Section 8) and deployment (Section 9). Each requirement names the design that meets it and the test that proves it.

4. Architecture

4.1 Context diagram (C4 Level 1)

The context view sets the system's boundary. Everything the system knows about the building comes through BuildSim's REST API. Everything it does to the building goes back the same way. The operator uses a browser. The occupants are simulated inside the system. I show them here as the people the system is for, because the safety rules (interlocks) exist to protect them. There is one piece of outside compute, and it is optional: the language model endpoint. When that detector is selected, the service asks it about one room at a time. The other candidate was the overnight model training from the proposal, and I did not need it. Nothing the system does to the building depends on the endpoint answering.

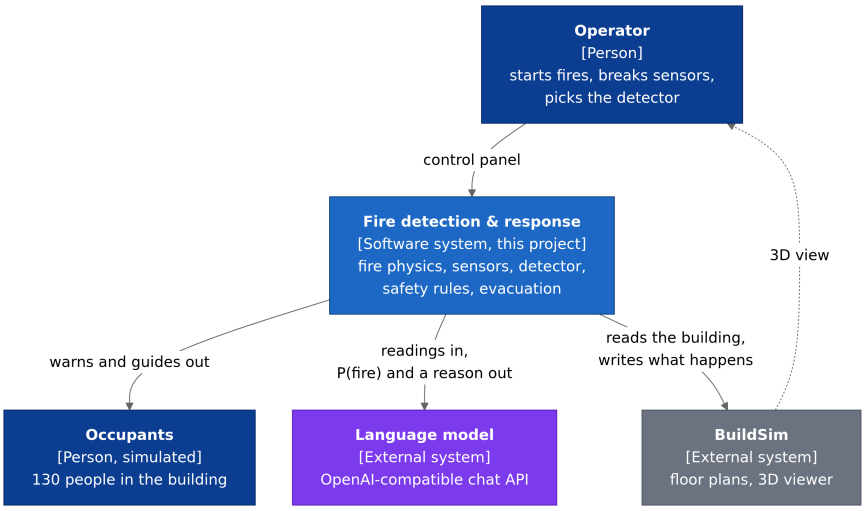


Figure 2: *C4 System Context (Level 1). The system in scope is blue. BuildSim (grey) and the language model (purple) are outside it. Every arrow says what it carries. The model only ever gets readings, never the truth.*

4.2 Container diagram (C4 Level 2)

The container view opens the box. Five parts run separately, and two of them are my code: the control panel in the browser and the firelab service. BuildSim and its 3D viewer are provided. The fifth, the model endpoint, is the only one not on the laptop. It is also the only one the system runs fine without.

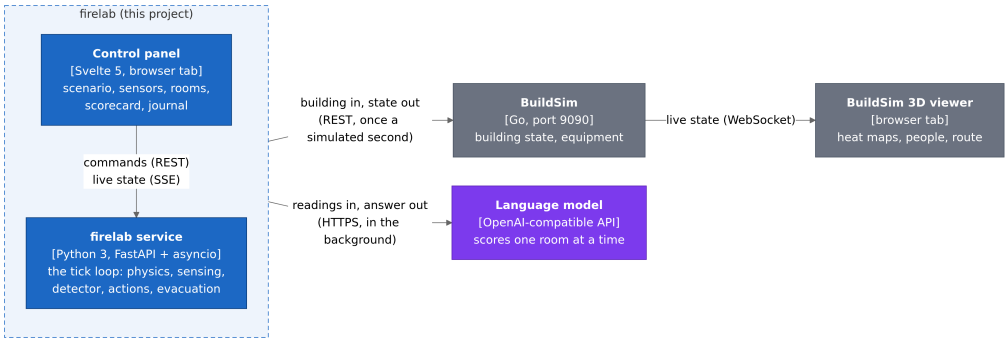


Figure 3: *C4 Container diagram (Level 2). The dashed boundary is the system. BuildSim and its viewer are grey, and the language model is purple. Every arrow shows its protocol. A background task calls the model, never the tick.*

Why one service. In the proposal, nine containers would each have held one of the four zones. I did not build it that way, for one reason: simulated time. The whole point of the system is to run a fire at forty times real time. A 25-second waiting time must still mean 25 simulated seconds. With a broker, every message would need a simulated timestamp. Every subscriber would have to wait for every publisher before moving on. A slow dashboard could stall the physics. Inside one process, the tick loop does four things in a fixed order under one lock. It moves the world forward, samples the sensors, thinks and acts. "Now" is one number. The price is that the zones share a process, so a crash in one takes down the others. Section 13 says what that costs and what would change if the system left the laptop.

How one process does several things at once. I built the service on Python's `asyncio`. Think of it as one worker that juggles many small jobs. Whenever a job has to wait, the worker puts it down and picks up another. Most of the work here is waiting: for BuildSim to reply, for the model to reply, or for a browser to collect the next update. Four jobs share the worker:

- the tick, which wakes every 250 ms and moves the fire forward;
- the web API, which answers the control panel and streams live updates to each open browser tab;

- the publisher, which sends all its updates to BuildSim at once (six fixed ones plus one per sensor), so it only waits as long as the slowest one;
- the model's helper, which asks the language model its questions in the background.

I chose one worker instead of several threads because two jobs can then never change the same data at the same moment. A job can only be paused at a point where it waits. The tick also holds a lock while it runs, so a button press can never change the building halfway through a tick. The downside is that a slow job that never waits holds everyone up. That is why the tick must finish within 250 ms, and why experiment E5 measures how many rooms fit in that time.

What keeps the zones apart is the package layering, since there are no process boundaries. There are three packages. `domain` is pure: standard library only, no clock, no network. `adapters` holds everything that speaks HTTP to BuildSim. `app` holds the tick loop, the API and the snapshot. Dependencies only point inward. A test, `test_layering`, fails the build if any module imports the wrong way. It also fails if anything in the intelligence path touches the true room state.

The control panel is a separate container because it has a different lifecycle. I build it once with Vite into static files, and the service serves them. It talks to the service only through the REST endpoints and the server-sent event stream. A script would use the same interface. The panel knows nothing about the engine's internals.

The model endpoint is outside the boundary for the opposite reason. It is not mine, and other users share it. It answers in about eight real seconds, while the tick has 250 milliseconds. Those two facts shaped the adapter in front of it. The service never waits for the model and asks it about one room at a time. It uses an answer for at most a minute of simulated time. It also slows the clock to what the measured answer time allows, so answers do not go stale (Section 7.1). I chose an OpenAI-compatible chat API because every hosted and local model server speaks it. So the endpoint, the key and the model name are three environment variables, and nothing in the code names a vendor.

4.3 Component diagram (C4 Level 3): inside the firelab service

The service is the one container complex enough to need its own picture. Its components are the four zones of one tick, plus the parts around them. The three groups mark the truth boundary. Components in the orange group may read the real state of the rooms. Components in the two blue groups may not. The only thing that leaves the orange group is a sensor reading. I put the LLM adapter in the readings-only group on purpose. It speaks HTTPS, so by the layering rules it lives under `adapters`. But it sends the same feature vector the rule sees. The layering test checks that it imports nothing that could tell it the truth.

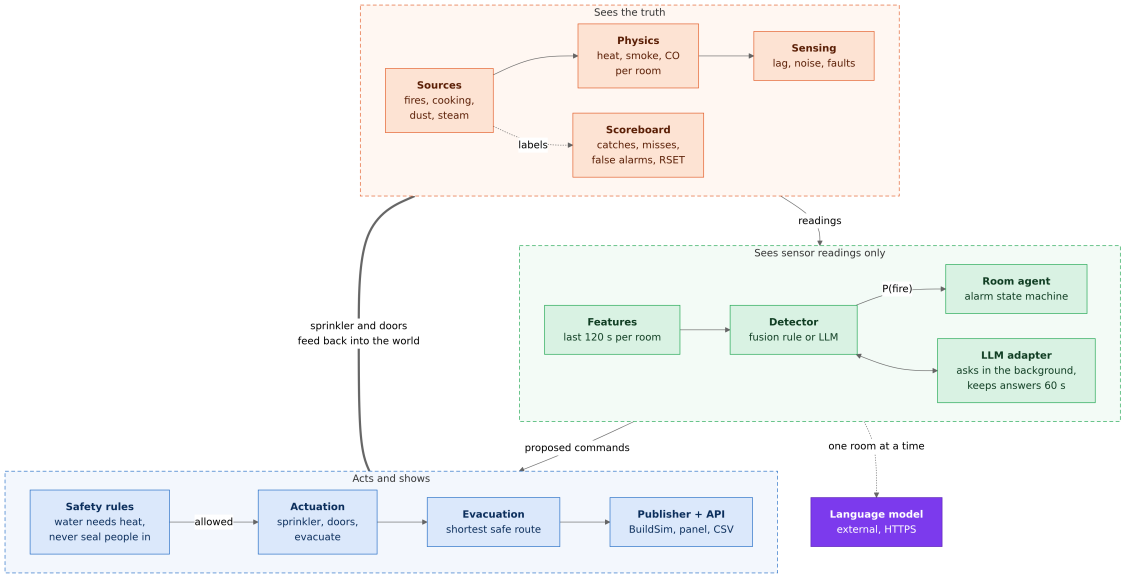


Figure 4: *C4 Component diagram (Level 3) of the firelab service. The groups run in this order every tick: the orange group (the world and the sensors), then the green group (the detector and the agent), then the blue group (the actions). Only readings leave the orange group, and the layering test fails if anything else does. The thick line from the blue group back to the orange one is the sprinkler and door state being written into the world. The dotted arrow is the one call that leaves the process: the LLM adapter asking the language model about a room, outside the tick.*

Component responsibilities. Each component has one job. The last column says why it is a separate component.

Component	Responsibility (one job)	Module	Reason for separation
Sources	What a fire, or cooking, dust or steam, gives off over time. Carries the ground-truth label	`domain/sources.py`	The label must never leak: only the scoreboard reads `kind`
World / physics	One lumped node per room. Each node settles towards the level its sources drive it to, and spreads through doors and stairs	`domain/physics.py`	Owns every time constant, so each source stays a few lines long
Sensing	Turns the truth into a reading through the corruption chain. Holds faults	`domain/sensing.py`	The line between truth and what is observed. The one place noise and faults live
Features	A 120 s window per room: rate of rise, ratio, neighbour agreement	`domain/features.py`	Sees readings only. This is the detector's input contract
Detector	Turns features into P(fire), the estimated chance that the room is on fire, with a contribution per feature. The fusion rule (my hand-written rule that combines the sensor features into one score) is one implementation	`domain/detector.py`	A protocol, so a second detector can replace the rule without touching the agent. One did
LLM adapter	The same protocol, answered from a cache of the model's answers. Asks in the background. The rule covers any room without a fresh answer	`adapters/llm/`	The network must never hold up the tick. One file per job (prompt, verdict, policy, client, detector), so each can be tested without an endpoint
Room agent	The alarm ladder with waiting times. Proposes commands with reasons	`domain/agent.py`	Decides but never acts. Testable with no actuators
Interlocks	Allows or refuses a command and says why	`domain/interlocks.py`	Kept outside the agent, so even a badly trained agent cannot trap anyone
Actuation	The only door between deciding and doing. Writes sprinkler and door state into the world	`app/engine/actuation.py`	Closes the loop. One audit point for every change

Component	Responsibility (one job)	Module	Reason for separation
Evacuation	Routes people around danger, walks them, and picks the one route to draw	`app/evacuation.py`, `adapters/walkways.py`	The one domain-level job that needs a route lookup
Scoreboard	Detections, misses, false alarms, RSET (the time people need to get out safely) and tenability (whether a room is still survivable)	`domain/scoring.py`, `timeline.py`, `tenability.py`	Grades the run. The only reader of the truth on the right-hand side
Publisher + API	Turns engine state into BuildSim payloads and the UI snapshot. REST, SSE, CSV	`adapters/publisher/`, `app/api/`, `app/snapshot/`	Keeps presentation out of the loop. BuildSim's payload shapes live in one place

4.4 Design decisions and justifications

Each row names the option I took, the alternative I considered, the requirement it serves and the price paid. A decision with no rejected alternative would only be an assumption.

Decision	Chosen	Alternatives	Justification (requirement)	Trade-off
Process model	One tick loop, four zones, one lock	One container per zone over MQTT (the proposal)	Waiting times and rates need one simulated clock. At 40×, a reading sent through a broker is stale when it arrives (NFR-01, NFR-03)	No fault isolation between zones. Splitting them is a future step
Concurrency	`asyncio`: one worker that switches between jobs while they wait	Several threads; one process per job	The work is mostly waiting on the network. With one worker, two jobs never change the same data at once, and the tick's lock covers the rest (NFR-01, NFR-03)	A slow job that never waits holds everyone up, so the tick must finish within 250 ms
Zone separation	Import layering checked by a test	Trust the code review	The truth is one attribute away from the detector. A test lets me check that the code keeps away from it (NFR-04)	A test to maintain as modules move
Detector	Hand-written fusion rule behind a protocol, as the default	Isolation forest; autoencoder; the model alone	It caught every fire, with zero false alarms at every noise level. It explains itself term by term and costs nothing per room (FR-01, FR-03, FR-06)	Weights tuned by hand. No learning from data
Second detector	A language model over an OpenAI-compatible API, given readings only	Isolation forest trained on exported runs; no second detector	This is the comparison the proposal promised, without training on the simulator's own output. The answer comes with evidence and a reason, so FR-06 still holds (FR-08)	About 8 s per answer, not repeatable, and an outside dependency
Model off the tick	Background worker, cached answers good for 60 s, rule fallback	Ask and wait for the reply; use stale answers	A tick has 250 ms and an answer takes seconds. A room must always have a number, and a stale number is a wrong one (NFR-06, NFR-07)	Two detectors are in play at once, and the journal has to say which one is speaking
Pacing	Cap the speed dial at what measured answer times allow	Let the clock run and accept that answers expire	At 40×, every answer arrived 320 simulated seconds late, and the rule quietly did all the work. A capped dial is honest (NFR-07)	With this endpoint, the demo runs at 3× instead of 40×
Questions in flight	One at a time, most suspicious room first	Several at once	The endpoint answers requests one after another. Two in flight doubled the delay and halved the speed the clock could run at. The fire room gets its answer first	The rule scores most of the quiet neighbours
Alarm logic	State machine with waiting times	Threshold on P(fire)	A single threshold flickers with noise. Waiting times cost seconds, but they stop doors opening and closing over and over (FR-03)	Adds 45 s to confirming a clear fire
Safety	Safety rules outside the agent	Rules inside the agent's policy	Whoever is wrong upstream, the building stays safe. The same check applies to a person's click (SR-01 to SR-03)	A confirmed fire in an occupied room gets no door

Decision	Chosen	Alternatives	Justification (requirement)	Trade-off
Deferred commands	A refused request stands until it can be carried out	Refuse once and forget	"Not yet" must not become "never". In the smoulder, the sprinkler was refused at 201 s and released at 219 s, once heat arrived (SR-01)	Re-checked every tick
Route search	My own Dijkstra search over BuildSim's graph, with alarming rooms removed	Ask BuildSim's router for each exit	BuildSim only returns the shortest path. A fire in a corridor makes every shortest path run through it (FR-05)	A second copy of the graph to keep consistent
Sensor emulation	Lag, bias, noise, quantisation, clamp, faults per device	Truth plus Gaussian noise	The interesting failures (saturation, stuck, dead) are not noise. The temperature clamp at 250 °C is what a real heat detector does (NFR-02)	More parameters to defend
Physics	Lumped node per room, first-order relaxation, door-gated diffusion	CFD; zone model with layers	Enough structure to close the loop and get the trends right. Cheap enough for 956 rooms 40 times a second (NFR-01)	The fire room overheats when sealed. No oxygen limit (Section 13)
Storage	In-memory rolling windows; CSV export	Time-series database; ThingsBoard	The detector may only see 120 s, and the charts only need 20 minutes. A database would be one more process to keep in step with the clock (NFR-01)	History is lost on restart. Long runs are exported, not stored
UI transport	Server-sent events, one snapshot per tick	Polling; WebSocket	One-way push with automatic reconnect, and a plain HTTP stream any tool can read	The snapshot is rebuilt every tick, even with no listener
BuildSim writes	Once per simulated second, complete collections, 24 sensor values per publish	Every tick; deltas	BuildSim replaces a whole collection on each write. Too many writes overloaded it (NFR-05)	Sensor labels in the tree lag behind the truth

5. Interfaces and communication

I use four patterns, each where it fits. **REST** carries every command and every call to BuildSim. Each is a one-to-one request, and the caller must know the result. For example, a refused command comes back with the safety rule's reason. **Server-sent events** carry the snapshot to the control panel. That flow is one-way and high-rate, and it needs no reply. **WebSocket** is BuildSim's own channel to its viewer. I use it, but I did not build it. **HTTPS chat completions** carry one room's readings to the language model and a JSON answer back. A background task makes the call, with a 30 s timeout and one retry. The result goes into a cache, never into a tick. There is no pub/sub, for the reason given in Section 4.2.

Naming. Everywhere, a room's address is `level/name`, for instance `level2/A3505`. I do this because a bare name is not unique across storeys. A device is `fl-<level>-<room>-<modality>`, so you can read its room and sensor type off its ID. Units are fixed and never sent in messages: temperature in °C, smoke as optical obscuration in 1/m, CO in ppm, distances on the plan in half-metre units, and simulated time in seconds since midnight.

The control-panel API. Every request body is a typed model. The framework rejects unknown keys before the engine sees them.

Method	Path	Body	Semantics
GET	`/api/state`		The full snapshot, once
GET	`/api/events`		SSE: one snapshot per tick, 4 Hz
GET	`/api/rooms`		Every space for the pickers
GET, PUT	`/api/config`	partial config	Live settings: speed, thresholds, waiting times, noise, population. `detector.kind` swaps the detector on the next tick
POST	`/api/clock`	`{seconds?, factor?, running?}`	Play, pause, jump, 0.1–600×
POST	`/api/scenario/ignite`	`{space, kind, growth, delay, peak_kw}`	Schedule a source. `kind` is also the label

Method	Path	Body	Semantics
POST	`/api/scenario/preset`	`{preset, space}`	Put sensors in the room and its neighbours, then ignite
DELETE	`/api/scenario/{id}`		Put a source out. What it gave off still spreads
POST	`/api/sensors/deploy`	`{spaces, modalities}`	Install devices. A room without devices is not monitored
POST	`/api/sensors/fault`	`{device_id, fault}`	`stuck | dropout | drift | dead | none`
POST	`/api/agent/mode`	`{auto}`	`false` means supervised: decisions are shown, not carried out
POST	`/api/actuators/command`	`{kind, space, value}`	A manual command. It passes the same safety rules and returns `{allowed, reason}`
GET	`/api/history?space=`		The last 240 samples of one room
GET	`/api/export.csv`		Truth beside reading beside P(fire), every room, labelled
POST	`/api/reset`, `/api/reload`		Quiet building. Reload floors from BuildSim

The snapshot is the one data structure the panel reads. Each monitored room appears as shown below. The three columns of the Rooms table are the three objects in it: what is true, what was reported, and what the system believes.

```
{"key": "level2/A3505", "level": "level2", "name": "A3505", "kind": "room",
 "truth":   {"temperature": 648.0, "smoke": 0.668, "co": 107.0},
 "reading": {"temperature": 250.0, "smoke": 0.665, "co": 101.0},
 "p_fire": 0.994, "state": "CONFIRMED", "tenable": false,
 "sprinkler": true, "door": "closed", "occupants": 0,
 "devices": [{"id": "fl-level2-A3505-smoke", "modality": "smoke",
              "fault": "none", "reading": 0.665}, ...]}
```

The temperature reading of 250 against a truth of 648 is not a bug in the example. It is the heat detector's range clamp. The snapshot exists to make this kind of thing visible.

The BuildSim interface. I publish truth and belief to different endpoints. So a false positive shows on screen as a room glowing red on the belief layer while it is cool on the heat map.

Endpoint	Direction	Carries	When
`GET /api/building/floors/{level}`	read	rooms, polygons, walkable graph	once, at load
`GET /api/building/cross-floor-edges`	read	stairs and lifts joining storeys	once, at load
`GET /api/graph/route`	read	a shortest path, multi-floor	fallback when no clear route exists
`PUT /api/room-layers`	write	true temperature, smoke, CO; and P(fire) as its own layer	every simulated second
`PUT /api/effects`	write	fire, smoke and sprinkler particles	every simulated second
`PUT /api/alerts`	write	the agent's alarm markers with P(fire)	every simulated second
`PUT /api/doors`	write	fire-door state. Also blocks the walkable router	every simulated second
`PUT /api/entities`, `/api/occupancy`	write	every occupant's position and figure; head count per room	every simulated second
`PUT /api/sensors/{id}/value`	write	one device's latest reading, into the equipment tree	24 per publish, round robin
`PUT /api/sessions/{id}/highlights`	write	red, orange, yellow outlines by alarm state	when they change
`PUT /api/sessions/{id}/route`	write	the one escape route drawn, each node with its storey	when it changes

A route is a list of waypoints plus its length in metres. Each waypoint carries its level, so a path down the

stairs is drawn on each storey it crosses. A closed door goes to BuildSim as well as into the physics. That way BuildSim's own router, which the viewer's route finder uses, stops sending people into a sealed room.

The model interface. There is one request per room: `POST /chat/completions` on whatever base URL is configured. It holds the fixed system prompt and the room's readings as plain lines with units. The prompt is the whole contract. It tells the model that it sees only the sensors, never the room. It also lists what separates a fire from cooking, dust or steam, in the same terms the rule uses. The reply must be one JSON object:

```
{"p_fire": 0.92,
 "evidence": [{"name": "CO 95 ppm", "weight": 0.30},
              {"name": "smoke rising", "weight": 0.25}, ...],
 "reason": "Smoke, CO and temperature are rising together."}
```

The service refuses any reply that does not parse as that, or whose probability is not a number. It does not try to repair it, and the room stays with the rule. The evidence becomes the "Why" bars on the panel. Next to them is a bar for the answer itself, on the rule's logit scale. That way the two detectors read the same way on screen.

6. Simulating the sensor values (the physical model)

BuildSim provides the rooms but nothing that changes. So I simulate the fire, the air in every room, the sensors and the people here. I never aimed for physical accuracy. I wanted a model with enough structure that a decision based on its readings means something. Fires keep growing. Cooking, dust and steam peak and fade. Smoke leaves through doors and climbs stairwells. Above all, actuator commands change the next reading, so the loop closes.

Sources. A source is a scripted event with a kind, a room and a start time. The kind is also the true label that the scoreboard uses. Nothing after the sensors may read it. Each kind states what it gives off at time e after ignition. This is a heat release rate, plus the smoke and CO levels the room would settle at if nothing else happened:

Kind	Label	Emission	What makes it hard
flaming	fire	$q = \min(\alpha e^2, q_{\max})$ with the standard t^2 growth coefficients (slow 0.0029 to ultrafast 0.1876 kW/s^2); smoke $0.06\sqrt{q}$, CO $4\sqrt{q}$	The easy case: all sensors agree
smouldering	fire	3 kW, smoke 0.9 1/m and CO 900 ppm, all rising as $1 - e^{-e/300}$	Almost no heat, so the safety rules (interlocks) refuse sprinkler water
cooking	nuisance	12 kW, smoke 0.35, CO 25 ppm, as a bump that rises over 3 min and fades over 15	Warm and smoky, but the CO never comes
dust	nuisance	smoke 0.5, a 20 s bump; no heat, no CO	Fools a smoke detector on its own
steam	nuisance	smoke 0.7, a 30 s bump; no heat, no CO	Like dust, but denser and shorter

The shape over time is what matters. A real fire keeps getting worse. Cooking, dust and steam grow, hold and fade. The detector has to pick up that difference over time, together with the ratio of CO to smoke.

Room physics. I model every space as one lumped node. It has a true temperature T, a smoke obscuration S (how much the smoke blocks light) and a CO concentration C. Each step of Δt seconds has two phases. First, each room moves towards the level its own source pushes it to:

$$T \leftarrow T^* + (T - T^*)\, e^{-\Delta t/\tau_T}, \qquad T^* = T_{\text{out}} + \min\Big(140\,(q/V)^{2/3},\ 900\Big),$$

Smoke and CO move towards their own equilibrium values in the same first-order way. The time constants are $\tau_T = 90$ s, $\tau_S = 45$ s and $\tau_C = 60$ s. V is the room's volume: its floor area from the plan times a 3 m ceiling. So a lecture theatre dilutes a fire and a cupboard does not. The temperature rises less than linearly

with heat density, and it is capped at post-flashover temperatures. Then neighbouring rooms swap what they hold through every doorway and stairwell:

$$\Delta X_{a\to b} = \min\big(g\,k_X\,\Delta t,\ 0.5\big)\,(X_a - X_b), \qquad g = \text{conductance} \times \text{openness},$$

for $X \in \{T, S, C\}$, with diffusion rates $k_S = 0.35$, $k_C = 0.30$ and $k_T = 0.12$. The conductance of a doorway comes from BuildSim's walkable graph. It is 0.6 for a stairwell, and that is how smoke climbs floors. The share a room gives away in one step is capped across all its doorways together. Without the cap, a corridor with twenty doors would hand out several times what it holds. The building would then fill with smoke that nobody produced. That was a real bug. I found it by running the building at high speed.

Closing the loop. Two actuator states live inside the physics. An active sprinkler multiplies the room's heat release by 0.25 and its smoke equilibrium by 0.4. A closed fire door sets the openness of every link into that room to 0.15, because a real fire door leaks. The actuation zone writes both states, and the next physics step reads them. So a command issued at tick n changes the reading at tick $n+1$. Section 11 measures what this gains.

Sensors. A reading is never the truth. Every device applies the same chain of steps, in this order. Each step has its own parameters for each sensor type:

`lag` → `calibration bias` → `noise` → `quantisation` → `clamp` → `fault`

Modality	Lag τ	Noise σ	Quantum	Range	Interval	Unit
smoke	12 s	0.010	0.005	0–3	5 s	1/m
CO	25 s	3.0	1	0–2000	5 s	ppm
temperature	30 s	0.15	0.1	−20–250	5 s	°C

Temperature lags most because a heat detector has thermal mass. In a developed fire, the dashboard shows a reading of 250 while the truth is 648. That is the clamp at 250 °C: the detector is saturated, as a real one would be. One live setting scales the noise. It is 0 for a perfect instrument and 3 for a badly behaved one, and the evaluation sweeps it. A device can also be given one of four faults. With *stuck*, it repeats its last value. With *dropout*, it skips 40 % of samples. With *drift*, it adds an error that grows without limit. With *dead*, it reports nothing at all. Faults are set from the Rooms table. The two hardest presets are built on them.

People. I did not build the daily occupancy schedule from the proposal. Nothing in fire detection depends on how many people are in a room at nine in the morning. What matters is where people are when the alarm sounds. So I scatter the population once over the routable rooms, using a seeded random generator. The default is 100 students, 10 lecturers, 5 staff, 5 security guards and 10 visitors: 130 people in all. Each role walks at its own multiple of the configured 1.3 m/s. The difference is about knowing the building, not fitness. A guard who has drilled the routes is 15 % faster than a student. A visitor who has never been here is 15 % slower. Once evacuating, a person follows the waypoints of their route. Each person carries a fractional effective CO dose, following ISO 13571. This is a running total of the CO they have breathed in every room they pass through. Walking out through a smoke-filled corridor adds to it, and the dose stays with them afterwards. A room stops being escapable at 60 °C, or when smoke cuts visibility below 10 m. The scoreboard counts those rooms as lost.

7. Autonomous services and data engineering

7.1 The autonomous service

The intelligence runs once per tick over every monitored room. It has three steps, and each step sees only the output of the one before. Readings become features. Features become a probability. The probability drives a state machine. The middle step has two versions: a hand-written rule and a language model. The operator can switch between them while a fire is burning. The rule is the simplest thing I found that meets the requirements. I added the model to find out what a second opinion costs and what it gains. The end of this section explains how I keep it from slowing the loop down.

Features. Each monitored room keeps a rolling 120 s window of readings for each sensor type. From this window I take the latest smoke, CO and temperature. I also compute the smoke rate of rise per minute, and the temperature rise above the building's 20 °C baseline. Another feature is the ratio of CO to smoke. It is high for a smoulder and near zero for dust or steam. The last two features are the share of monitored neighbours that also see smoke, and the coverage. Coverage means how many sensor types reported at all. Nothing here reads a room's true state. The test suite fails if that ever changes.

The rule. The default detector is a hand-written fusion rule. A fusion rule combines several sensor readings into one score. Each feature adds a weighted term with an upper limit. The terms are summed with a negative bias. The sum then passes through a logistic function, which squeezes any number into a value between 0 and 1. The result, $P(\text{fire})$, is the detector's estimate of how likely it is that the room is on fire:

$$z = -3.4 + 3.2\,\min(S, 1.5) + 1.8\,\frac{\max(C-25,0)}{125} + 1.6\,\min\!\left(\frac{C/S}{250}, 2.5\right)\gamma + 0.55\,\frac{\min(\Delta T, 60)}{10} + 1.4\,\dot{S} + 0.8\,N,$$
$$P(\text{fire}) = \sigma(z)\cdot\left(0.55 + 0.45\,\frac{\min(\text{cov}, 3)}{3}\right),$$

where $\gamma = \min(S/0.15, 1)$ keeps the ratio term switched off until there is enough smoke to divide by. $\dot{S}$ is the clipped smoke rate and N is the neighbour agreement. The last factor holds the score back when fewer than three sensor types are reporting. CO only starts to count above 25 ppm, which is where cooking peaks. That is what separates a frying pan from a smoulder. Every term has an upper limit, so no single sensor type can push the sum to the top on its own. That is why a dead smoke detector does not blind the system. It is also why a runaway one does not set off the alarm (Section 11).

The detector is a protocol with two methods, `probability` and `explain`. The second one is not optional. The explanation is the calculation itself. The contributions are the terms above. The dashboard draws them as bars under the heading "Why 99 %". Nobody could sign off a detector that cannot say why. The model has to implement the same two methods, and it does.

The alarm ladder. The agent is one state machine per room. Only the probability and the clock drive it. The last column is the waiting time:

Transition	Condition	Dwell
NORMAL → INVESTIGATING	$P \geq 0.35$	none
INVESTIGATING → PRE-ALARM	$P \geq 0.55$ held	20 s
PRE-ALARM → CONFIRMED	$P \geq 0.75$ and in pre-alarm for	25 s
CONFIRMED → SUPPRESSED	sprinkler running and $P < 0.75$ for	25 s
any alarm → CLEARING	$P \leq 0.25$ held	90 s
CLEARING → NORMAL	quiet for	90 s

The waiting times are the defence against false alarms. A single threshold would flicker on sensor noise and open and close fire doors all day. Holding a level for a fixed time costs 45 s on a clear fire. In return, the system stays silent on cooking, dust and drifting sensors. Standing down takes far longer than raising the alarm did, and that is on purpose. The thresholds and waiting times are live settings, so you can explore the trade-off from the panel.

What it asks for. Only two states issue commands. Entering CONFIRMED asks for the sprinkler on, the fire door closed and the building evacuated. Entering CLEARING asks for the reverse. Every command carries a reason string, such as "P(fire)=0.91 in A3505", that goes into the journal. The agent never writes actuator state. Its requests go to the safety rules. If the safety rules refuse a request, it stays pending and is offered again every tick. "Not yet" must not become "never". In the smouldering run, the sprinkler was refused at 201 s because there was not enough heat. It was released at 219 s, when the room passed 35 °C. Nobody had to ask twice.

Failure and safety. If the intelligence is wrong, the safety rules still hold. There is no water without heat. No door is ever locked. No door closes on an occupied room or an escape route. If the intelligence is unavailable, nothing moves. Actuators keep their last state, and the building is as safe as it was without the system. If a person disagrees, supervised mode shows every decision but carries out none. A manual command from the panel passes through the same safety rules. It is logged in the journal with the same reasons. If a room has no working sensor, the detector gives no opinion rather than a low one. So a room

without sensors is never marked safe. None of this changes when the model is the detector. That is why I kept the safety rules outside the intelligence in the first place.

The second detector: a language model. The proposal planned to compare the rule with an isolation forest, a standard machine-learning method for spotting unusual data. I built the comparison with a language model instead, for two reasons. First, the only labelled data to train a forest on would have been this simulator's own output. The forest would have learned my physics, not fires. Second, a model that answers in words can be asked to explain itself. Trained detectors usually fail that requirement. The adapter in `adapters/llm/` implements the same two-method protocol as the rule. It sends one room's readings to any OpenAI-compatible chat endpoint. It sends the same features the rule sees, and nothing else. The system prompt tells the model that it sees only the sensors, never the room. It also lists what separates a fire from cooking, dust or steam, in the same terms the rule uses. A flaming fire gets worse on all three sensor types. A smoulder is dense smoke and high CO with no heat. Cooking is warm and smoky, with CO under 25 ppm. Dust and steam are smoke alone. A lone sensor that disagrees with its neighbours is probably broken. The reply is one JSON object with a probability, a list of weighted evidence and a one-sentence reason. The adapter parses it strictly. It refuses anything malformed instead of repairing it.

The real engineering is around the call. The tick has 250 ms, and the endpoint I had takes about eight seconds to answer. Figure 5 shows the setup. `probability()` never waits. If the model's last answer for the room is younger than 60 simulated seconds, it returns that answer. Otherwise it returns the rule's number. At most, it queues a question. A worker task runs beside the tick loop and sends the queued questions one at a time. The most suspicious room goes first, ranked by the rule's own score. Each room is asked at most once every ten simulated seconds. A room whose sensors show nothing is never asked, so a quiet building costs nothing. Each answer is stamped with the time its readings were taken, not the time it arrived. That is what makes the 60 s expiry mean something. The journal records each hand-over in both directions. It also records every answer that moves the score by a tenth, together with the model's reason.

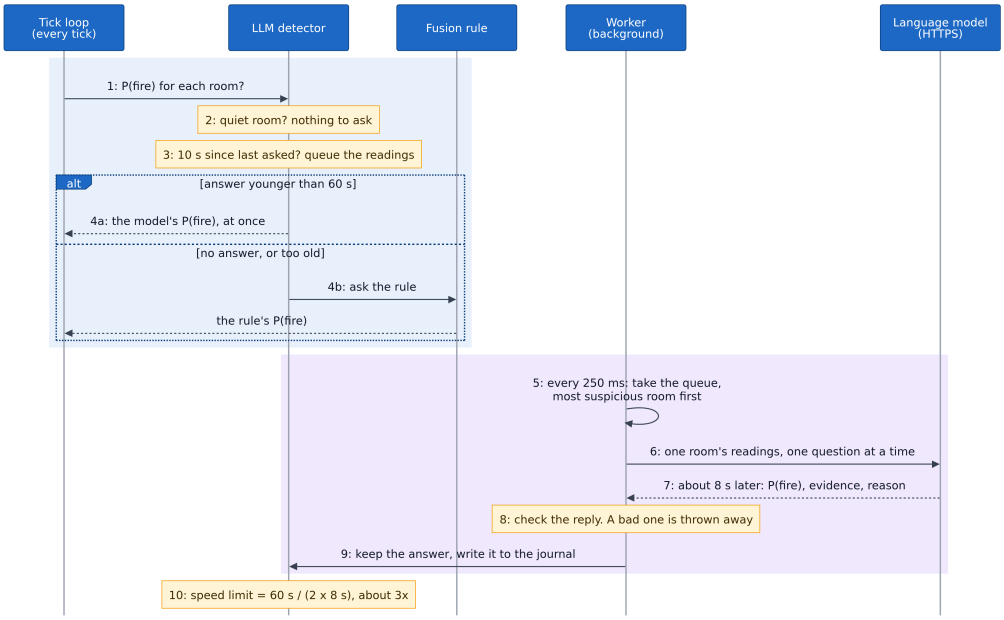


Figure 5: *How the language model scores a room without holding up the tick. The tick only ever reads a cache or the rule. The worker fills the cache in the background. I measured an answer time of about 8 s, and this caps the clock near 3× real time.*

Pacing. The first time I ran the model at forty times real time, nothing happened. Every answer arrived 320 simulated seconds after the question. Every answer had expired by the time it arrived. So the rule scored every room, while the panel said "llm". This is the failure I had to design against. The fix is not a longer expiry. An answer a minute old is about a room that has since doubled its smoke. Instead, the detector measures how long answers take. It then reports the highest speed at which an answer still arrives with half its life left, $60/(2 \times \text{latency})$. Every tick, the engine caps the speed dial there and writes one journal line

about it. With the endpoint I used, that is 3×. The endpoint answers requests one after another. So asking about two rooms at once only doubled the latency and halved the cap. That is why only one question is in flight at a time, and the fire room gets it. Switch back to the rule and the cap lifts.

What the comparison is for. The rule is the baseline that the model has to beat on three counts. Does it catch the same fires and ignore the same cooking, dust and steam? How much later does it act? Is its "Why" as easy to check as a sum of terms? Section 11 reports all three. The model's evidence is drawn as the same bars. There is one extra bar for the model's answer itself, on the rule's logit scale (the scale of z before the logistic function). So the two are judged on the same screen.

7.2 The data pipeline

Readings flow through a short in-memory pipeline. It serves four consumers at once: the detector, the control panel, the scoreboard and the CSV export. It is small. The proposal included a time-series database and a message broker. I dropped both. Each one would have been another process to keep in step with a clock running forty times faster than real time.

- **Ingestion.** Every device samples the truth at its own interval. It appends the reading to its room's window. The window keeps 120 s of simulated time per sensor type, because that is all the detector may look at.
- **Features and score.** Once per tick, the features are extracted and the detector scores the room. The contributions are kept for the "Why" panel, and the agent takes a step.
- **The flight recorder.** Every 5 simulated seconds, it stores one row per monitored room. A row holds the time, the true temperature, smoke and CO, the three readings, and P(fire). Each room keeps its last 240 rows, about twenty minutes. This is what the charts show.
- **The journal.** It logs every state change, and every command with its reason. It logs every refusal with the safety rule's reason. It also logs every detector answer that moved a room's score by a tenth. That line holds the rule's top three terms or the model's reason. The journal keeps the last 200 lines. This is where the lessons show up.
- **The scoreboard.** It judges each new alarm once, against the source labels. The result is a catch with its latency, or a miss after 300 s without confirmation. It can also be a false alarm, together with the cooking, dust or steam that caused it. Smoke that came through the door from a real fire next door is not a false alarm. A fire on another floor is no excuse.
- **Out.** The snapshot is rebuilt once per tick and pushed to every open panel over SSE. The whole recorder, with labels, is one GET request away as CSV.

Data quality is handled at the source, not cleaned afterwards. The project exists to see what the intelligence does with bad data. A dead device stops contributing, and the coverage factor lowers the score. A dropout leaves gaps that the window tolerates. A stuck value flattens the rate term. Retention is short on purpose. Anything beyond twenty minutes is exported, not stored, and a restart starts clean.

8. Behaviour

8.1 Key scenario, dynamic diagram

Figure 6 is a C4 dynamic diagram of the office fire from Section 2. It uses the times measured in the run reported in Section 11. It shows what the static diagrams cannot: the tick order, the waiting times, and the moment the loop closes.

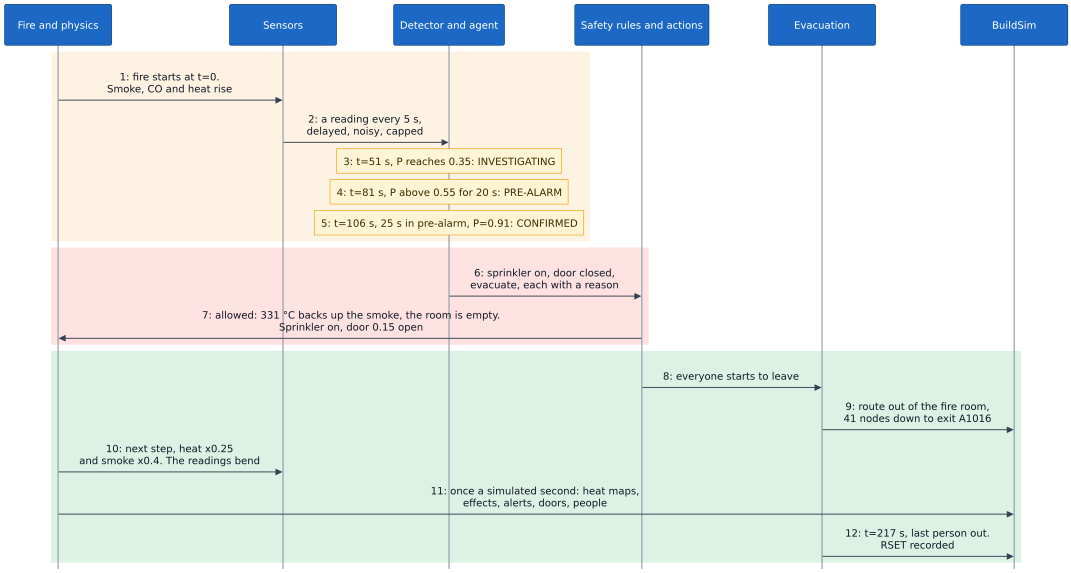


Figure 6: *C4 Dynamic diagram of the office fire, with the times measured in Section 11. Steps 3 to 5 are the waiting times. Step 7 is where the safety rules decide. Step 10 is where the loop closes.*

Two things in the order matter. First, the response is issued and applied in the same tick that confirms the fire. Nothing waits for the next reading. Second, the fire door closed only because A3505 happened to be empty and on nobody's route. If a student had been inside, step 7 would have refused to close the door. It would have logged why in the journal and kept the request pending until they left.

8.2 Process lifecycle, state machine

Every monitored room has one of these state machines. Its transitions are the code in `domain/agent.py`. The waiting times are on the arrows.

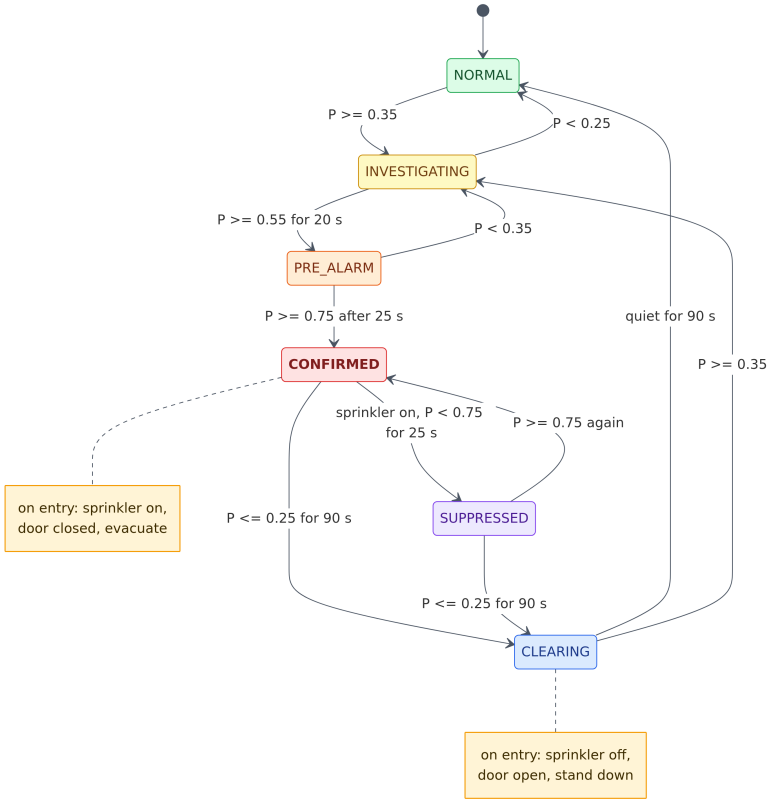


Figure 7: *The per-room alarm state machine. Only two states ask for anything. Every other arrow only changes what the room believes.*

SUPPRESSED is still an alarm. The fire is being held, not put out, and the requests from the confirmed state still stand. A fire that keeps burning through the water climbs straight back to CONFIRMED. To stand down, the probability must stay at or below 0.25 for 90 s. Then the room needs a further 90 s of quiet before it stops being flagged as an incident. So a sensor that recovers for a moment cannot reopen a door onto a fire. The process also has its own lifecycle, outside this diagram. On startup, the service loads the floors from BuildSim. It installs the detector named in the configuration. If that detector is the model, it also starts the model's worker task. Then it scatters the population. If BuildSim stops answering, the service does not crash. It marks itself disconnected, keeps simulating, stops publishing, and waits for a reload. Changing the detector while running stops one worker and starts the other under the tick lock. So no tick ever sees half a swap. The rooms' alarm states carry over untouched, because the agent never knew which detector it was listening to.

9. Deployment and component placement

Everything that matters runs on one laptop. So deployment is about how the system is built from processes, not where they sit in the world. There are two processes and two browser tabs. They talk only through the interfaces of Section 5. One optional call leaves the machine. When the language model is the chosen detector, it runs on a shared server on the university network.

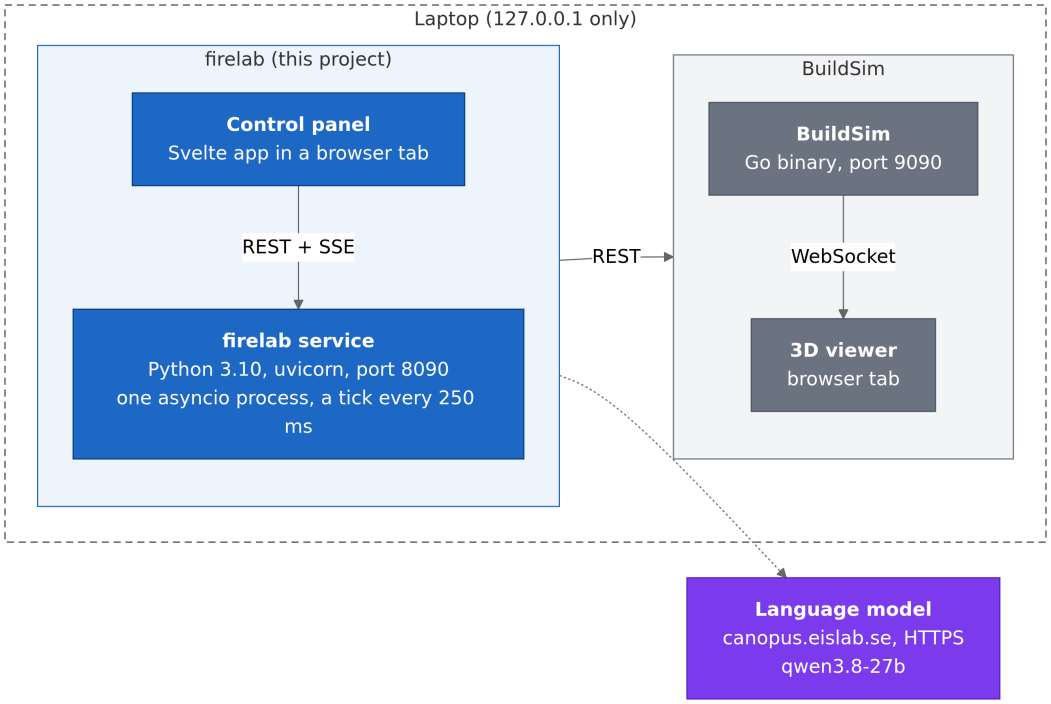


Figure 8: *Deployment view: two processes, two tabs, loopback only. The purple box, reached by the dotted arrow, is the only part that leaves the laptop. The system keeps detecting without it.*

Why this placement. The control loop lives in the firelab process. It runs from the physics, through the sensors, to the safety rules (interlocks) and back. It touches nothing outside the machine except BuildSim over loopback. In a real building, this process is the edge node. It must keep working when the internet does not. It must also answer within the tick budget, which Section 11 measures. The model endpoint puts that idea to the test. It is off the edge, it is slow, and someone else runs it. So the adapter treats it as advice, not as something the system depends on. The rule scores every room the model has not answered for. The tick never waits for the call. If the endpoint disappears, the journal says so and nothing else changes. A real building would need this design for any cloud service on the detection path. That is why the box is dotted. BuildSim stands in for the building's own controllers, and I left it alone.

What is lost by merging the zones into one process, since the course asks: fault isolation and independent restart. If the physics crashes, the agent dies with it. That is fine for a simulator on a laptop, but not for a building. The safeguard I built in is the layering. It keeps the zones as easy to separate as separate processes would. I also know where to split. The sensing zone already produces readings per device, keyed by device ID. The actuation zone already takes commands with a reason. Those are the two messages a broker would carry.

Operational notes. You start the service with `make run`. This creates a virtual environment, builds the panel and serves it. Configuration is live. Speed, thresholds, waiting times, sensor noise and population change on the next tick through `PUT /api/config`. A reload re-reads the floors from BuildSim without a restart. If BuildSim is down at startup, the service says so in its journal and sits idle until a reload. If BuildSim goes away mid-run, the service marks itself disconnected and stops writing. It keeps simulating.

10. Test plan

Two kinds of evidence back the requirements. The first is the unit suite: 162 tests and 98 subtests in 22 files. It runs in three seconds with nothing else running. That works because the domain imports nothing that needs a network. The model detector is tested against a fake endpoint and a fake clock. The second is the experiments (E1 to E7). They drive the real engine tick by tick against the loaded building, with publishing switched off. They use the same seed every time and write their measurements to `experiments/results/`. E7,

the model comparison, is the only one that also needs the endpoint and a wall clock. Section 11 reports them.

ID	Test	Requirement	Evidence
T-1	`test_agent`: the ladder climbs one step at a time and respects every waiting time. Only CONFIRMED and CLEARING send commands	FR-01, FR-04	unit, passes
T-2	`test_detector`: P(fire), the estimated chance of fire in a room, equals the sum of its parts. A room with no sensors gives no opinion. Cooking scores below the confirm threshold	FR-03, FR-06	unit, passes
T-3	`test_interlocks`: water is refused below 35 °C. Lock is always refused. Close is refused when the room is occupied or on a route	SR-01 to SR-03	unit, passes
T-4	`test_sensing`: lag, rounding (quantisation), clamping and each of the four faults behave as specified	NFR-02	unit, passes
T-5	`test_physics`, `test_sources`: relaxation, the diffusion cap, sprinkler factors, the t^2 curve	Section 6	unit, passes
T-6	`test_evacuation`, `test_walkways`, `test_multifloor`: clear routes avoid danger, upper floors reach a ground-floor exit, and the drawn route starts at the fire	FR-05	unit, passes
T-7	`test_scoring`, `test_history`: the catch, miss and false-alarm rules. The RSET milestones (RSET is the time from ignition until the last person is out). The recorder's columns and length	FR-07	unit, passes
T-8	`test_layering`: the direction of imports and the truth boundary	NFR-04	unit, passes
T-9	`test_publishing_route`, `test_publisher_doors`: the route is sent only when it changes, and it names its storeys. Closed doors reach BuildSim	NFR-05	unit, passes
T-10	`test_reset`, `test_presets`: a reset returns to a quiet building and forgets injected faults. Every preset is well formed	NFR-03	unit, passes
T-11	`test_llm_detector`: the prompt carries readings only. A good reply becomes an answer, and a bad one is refused. The rule answers until the model has spoken, and again once the model's answer expires. A quiet room costs no question. Each room gets one question at a time, the most suspicious first. The speed limit follows measured answer times	FR-06, FR-08, NFR-06, NFR-07	unit, passes
T-12	`test_pacing`: the rule has no speed limit. A slow detector caps the speed dial once and says so. Switching back lifts the cap	NFR-07	unit, passes
E1	Each of the six presets in A3505 for 20 simulated minutes: state times, score, response, peak truth	FR-01 to FR-04, SR-01	Table 10, Figure 9
E2	The office fire with the agent's commands dropped (supervised mode), then with the sprinkler alone	Section 6 loop	Table 11
E3	Office fire and kitchen cooking at sensor noise 0, 1, 2, 3×	FR-03, NFR-03	Table 12
E4	Office fire with the smoke detector stuck, dropping out, drifting, or dead	NFR-02	Table 13
E5	Sensors in 10 to 956 rooms; wall time of one tick at 1× and 40×	NFR-01	Figure 10
E6	Every experiment rerun from the same seed	NFR-03	identical latencies to the second
E7	The six presets, cooking at 2 and 3× noise, and the fire with a stuck smoke detector, scored by the language model at the speed it can keep up with	FR-08, NFR-06, NFR-07	Table 14, Figure 11

11. Evaluation and experiments

This section reports what I measured. The experiment harness in `experiments/run_experiments.py` measured every number below. It ran on the loaded A-building, with the fire in office A3505 on the top floor. There

were 130 people, placed by the default seed, and the thresholds were the defaults. The fire room and its neighbours had all three sensor types, as the presets do.

Detection and response (E1).

Preset	Investigating	Confirmed	Max P(fire)	False alarms	Sprinkler	RSET	Outcome
Office fire (flaming, fast)	51 s	106 s	0.994	0	on at 106 s	217 s	caught ✓
Overnight smoulder	136 s	201 s	0.991	0	refused 201 s, on 219 s	312 s	caught ✓
Kitchen cooking	326 s	never	0.688	0			correctly ignored ✓
Dust storm	never	never	0.129	0			correctly ignored ✓
Fire, dead smoke detector	61 s	never (room); 156 s (neighbour)	0.748	0	on next door	267 s	caught by the neighbour, see below
Drifting sensor, no fire	never	never	0.057	0			correctly ignored ✓

Table 10: *One run of each preset. Times are in seconds since ignition. RSET runs from ignition until the last of 130 people is out. In every caught case, the building was empty 111 s after the alarm.*

Both fires were confirmed well within the 180 s and 300 s limits of FR-01 and FR-02. None of the three no-fire presets (cooking, dust and the drifting sensor) raised an alarm (FR-03). Figure 9 shows the office fire as the system saw it. The truth and the reading agree until the heat detector maxes out at 250 °C. The probability climbs through the three thresholds. The waiting times show as the flat time between crossing a level and changing state. After the sprinkler starts at 106 s, the true smoke bends down. The temperature keeps rising in the now sealed room. The smoulder shows the safety rules at work. It was confirmed on CO with the room at 34 °C. Its sprinkler request was refused for lack of heat, and the journal logged this. The request was granted 18 s later, when the room passed 35 °C. The design worked as intended, but the number deserves a second look. A limit of 35 °C is a low bar for “heat corroboration” (backing from a heat reading). A warm summer office would pass it.

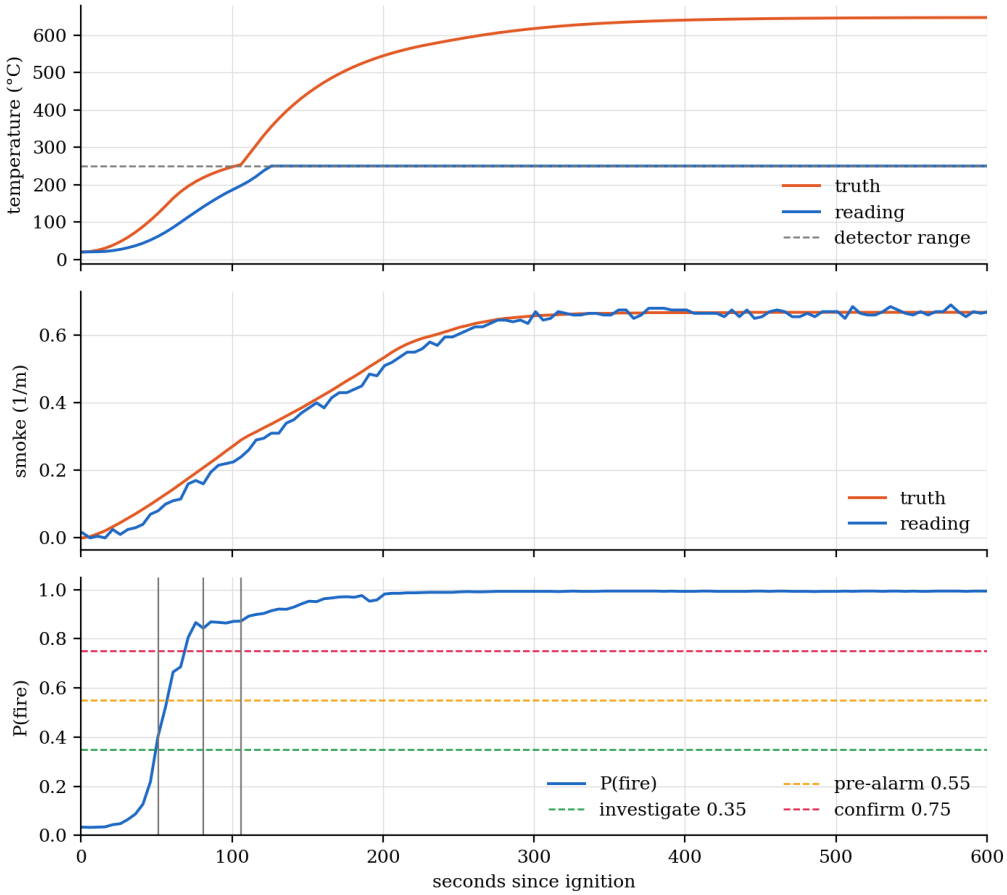


Figure 9: *The first ten minutes of the office fire in A3505, as the system saw it (E1). Top: the heat detector maxes out at 250 °C while the room reaches 648 °C. Middle: smoke. The reading follows the truth with a 12 s lag. Bottom: P(fire) passing the three thresholds. The vertical lines are the state changes at 51, 81 and 106 s. The flat stretches before them are the waiting times.*

The blind spot. The preset with a dead smoke detector is the one result that did not go to plan. Heat and CO alone pushed the fire room to a probability of 0.748. The coverage factor held it there. This factor caps a room with two working sensor types at 85 % of its logistic score. The confirm threshold is 0.75. So the room sat in PRE-ALARM for the whole run, three thousandths short. The building was still evacuated at 156 s. Smoke crossed into A3500A next door, and its three working detectors confirmed. The fire room also fed A3500A's neighbour-agreement term. I took two lessons from this. First, the neighbour cross-check is not decoration: it is what caught this fire. Second, the coverage penalty is tuned slightly too hard. Two changes would have confirmed the room itself. One is lowering the penalty by raising its floor from 0.55 to 0.60. The other is letting a neighbour's confirmation lift the room. This is the first change I would make.

Closing the loop, and what containment actually does (E2).

Response	Rooms lost	Neighbour peak smoke	Neighbour peak temp	Fire room peak temp	RSET
None (supervised, nobody presses)	5	0.382 1/m	149 °C	309 °C	no evacuation
Autonomous	4	0.171 1/m	152 °C	648 °C	217 s
Sprinkler only, door held open	5	0.171 1/m	152 °C	648 °C	217 s

Table 11: *The same office fire three times. "Rooms lost" are rooms that became untenable at any point in 20 minutes. Untenable means unsafe for people to stay in, judged by the limits in the ISO 13571 standard.*

The response halves the smoke that reaches the neighbours and saves one room. It also makes the fire room hotter. That surprised me until the journal explained it. With the door open, the fire room dumps heat into

five neighbours every step. With the doors closed, it keeps the heat. The sprinkler's 25 % factor does not make up for that. In this model, water reduces the heat the fire releases. It does nothing to cool the room directly. The third row tells me more. Holding the fire room's own door open made no difference at all. At 231 s the neighbour A3500A confirmed on the smoke that had crossed into it. It then closed *its* doors, and one of them is the same doorway. So in this building, containment depends on every room around the fire, not on the fire room alone. Without any response, nobody is told to leave and no RSET is recorded. That is the simplest argument for making the evacuate command automatic.

Noise (E3) and lying sensors (E4).

Noise	Office fire		Kitchen cooking	
	confirmed	max P	alarms	max P
0×	106 s	0.994	0	0.447
1×	106 s	0.994	0	0.688
2×	106 s	0.995	0	0.749
3×	106 s	0.996	0	0.741

Table 12: *Sensor noise sweep (E3).*

Smoke detector	Confirmed	Max P
healthy	106 s	0.994
dropout (40 % lost)	101 s	0.995
drift	106 s	0.995
stuck	156 s	0.880
dead	never (neighbour 156 s)	0.748

Table 13: *Faults on the fire room's smoke detector during the office fire (E4).*

Detection time did not move by a second across the noise sweep, and cooking never raised an alarm. That is the waiting times doing their job. The number to worry about is the cooking peak at 2× noise: 0.749 against a confirm threshold of 0.75. It did not confirm. To confirm, it would first have to hold 0.55 for 20 s and then sit 25 s in pre-alarm, and it never did. But the margin on the probability itself is only one noise sample wide. The CO term is what keeps cooking down. So, to be accurate, this detector tells cooking from fire by CO and time, not by any single reading. On faults, a detector that dropped readings caught the fire slightly faster. The readings it did send happened to land on rising edges. A stuck detector cost 50 s, because the rate term went flat. Drift did nothing in twenty minutes. A dead detector is the blind spot above.

Scaling and the breaking point (E5). The tick loop wakes every 250 ms of real time. It must finish before it wakes again. At 1× speed, one tick is one physics step over the whole building plus one intelligence pass over the monitored rooms. At 40× it is forty physics steps and one pass.

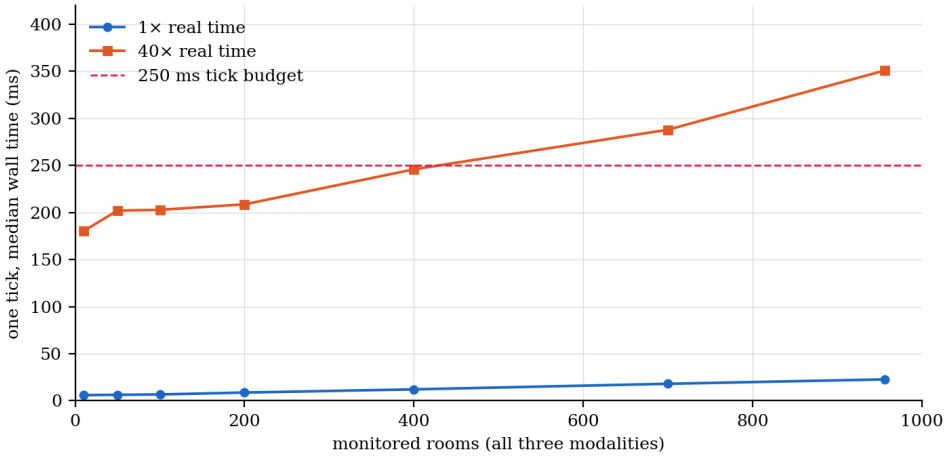


Figure 10: *Real (wall-clock) time of one engine tick against the number of monitored rooms (E5), on the laptop the demo runs on. The building has 956 rooms and 1005 doorways.*

At real time the whole building fits with room to spare: 23 ms per tick with all 956 rooms monitored and 2868 devices sampling. At 40× the tick goes over budget at about 450 monitored rooms. The shape of the curve shows where the cost is. With only 10 rooms monitored, a 40× tick already takes 180 ms. So the forty physics steps over the full graph are the fixed cost, about 4.5 ms a step. The intelligence adds about 0.18 ms per monitored room on top. The physics cost does not grow with the number of sensors. So for a

bigger building or a faster clock, the fix is a sparser diffusion step, not a faster detector. When the tick goes over budget, nothing breaks. The loop just runs slower than the speed dial claims. The tick also caps itself at sixty physics steps, so a stall cannot snowball.

The language model as the detector (E7). I ran the same scenarios again with the language model (`qwen3.8-27b` on the lab's server) deciding P(fire) instead of the rule. The script is `experiments/compare_detectors.py`. The model is asked about one room at a time, and each answer is trusted for 60 simulated seconds. When the model has no recent answer, the rule fills in. So the numbers below are for the model and the rule working together, not the model on its own. These runs are slow because they wait for real answers: the nine of them took 98 minutes.

Scenario	Rule: alarm	Model: alarm	Model max P	Model share	Answered / failed / asked	Median answer	Clock
Office fire	106 s	136 s	0.99	57 %	32 / 2 / 35	7.3 s	2.5×
Overnight smoulder	201 s	201 s	0.97	32 %	16 / 5 / 22	14.8 s	2.3×
Kitchen cooking, 1× noise	none	none	0.22	39 %	33 / 1 / 36	14.5 s	2.0×
Kitchen cooking, 2× noise	none	none	0.72	38 %	33 / 2 / 36	15.1 s	2.0×
Kitchen cooking, 3× noise	none	none	0.76	43 %	35 / 1 / 37	17.3 s	1.8×
Dust storm	none	none	0.07	92 %	36 / 0 / 36	7.2 s	3.0×
Fire, dead smoke detector	156 s (neighbour)	156 s (neighbour); room 798 s	0.90	18 %	22 / 3 / 26	21.5 s	2.1×
Drifting sensor, no fire	none	none	0.08	86 %	43 / 0 / 45	7.9 s	3.0×
Office fire, stuck smoke detector	156 s	156 s	0.93	57 %	37 / 4 / 42	20.0 s	1.4×

Table 14: *The rule compared with the language model (E7). Alarm: when the first room in the building was confirmed, in seconds after the fire started. Model share: how much of the time the model, not the rule, was scoring the fire room. Clock: how fast the simulation could run so the model's answers were still fresh when they arrived.*

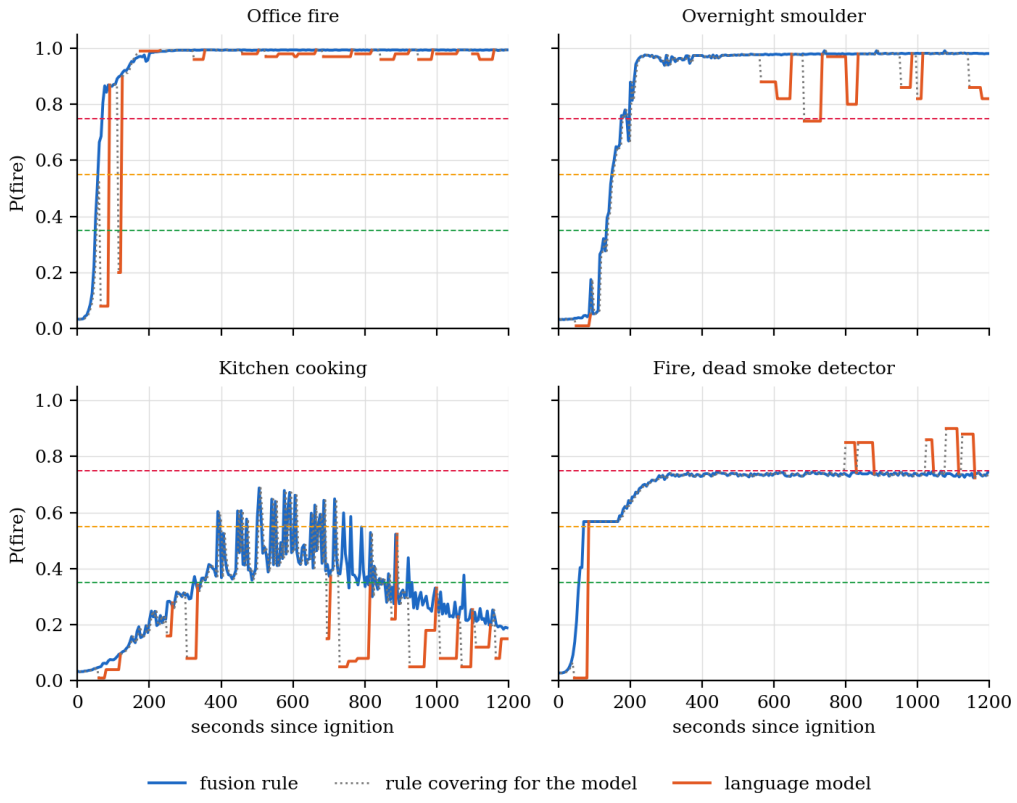


Figure 11: *P(fire) in the fire room with each detector (E7). Blue: the rule on its own. Orange: the model's answers. Grey dots: the rule filling in while the model had no recent answer. The dashed lines are the three alarm levels.*

In all nine runs the end result was the same as with the rule: every fire was caught and nothing else set off an alarm. The differences are in how fast, and in what it cost.

In the office fire, the model's first answers were low. Twice, at 64 s and 116 s, they sent the fire room back

to NORMAL, and the waiting time started over. The alarm came 30 s later than with the rule, and it came from the room next door (A3500A). The fire room itself was confirmed at 170 s.

With the dead smoke detector, the model did something the rule could not. It gave the fire room 0.85 to 0.90, while the rule got stuck at 0.748, so the model confirmed the room itself. But its answers about that room came so slowly that this only happened at 798 s. By then the room next door had already raised the alarm, at 156 s, in both runs.

For cooking, the model was calmer than the rule at normal noise (0.22 against 0.688). At three times the noise it was not: it reached 0.76, just over the confirm level. The waiting times stopped it from raising an alarm, just as they did for the rule.

The real cost is speed. A typical answer took 7 to 21 s, and the slowest took 85 s. 18 of 315 questions failed. The simulation could only run 1.4 to 3 times faster than real time, while the rule manages 40 times. In the quiet runs the model was scoring the fire room 86 to 92 % of the time, but in the fires only 18 to 57 %. So in the runs that mattered most, the rule did a lot of the work. The results also change from run to run, even with the same seed, because what the model is asked depends on how long its last answer took.

What the model gave me was a second opinion where the rule was too strict, and a reason in the journal that a person can read. What it could not do was make things worse than a delay. The waiting times, the safety rules and the rule filling in all sit between the model's number and anything that happens in the building.

Headline results.

Metric	Measured	Requirement	
Flaming fire confirmed	106 s	$\leq$ 180 s (FR-01)	✓
Smouldering fire confirmed	201 s	$\leq$ 300 s (FR-02)	✓
False alarms, 3 no-fire presets $\times$ 4 noise levels	0	0 (FR-03)	✓
Commands on confirmation	sprinkler, door, evacuate, same tick, with reasons	(FR-04)	✓
Sprinkler refused without heat	yes, released 18 s later	(SR-01)	✓
Fire caught with a stuck, dropping or drifting smoke detector	yes, worst 156 s	(NFR-02)	✓
Fire room confirmed with a dead smoke detector	no; caught next door at 156 s	(NFR-02)	partial
Tick at 1$\times$, full building	23 ms	$\leq$ 250 ms (NFR-01)	✓
Breaking point at 40$\times$	about 450 monitored rooms	measured (NFR-01)	✓
Building of 130 cleared after the alarm	111 s	reported (FR-07)	✓
Same seed, same run	identical to the second, every rerun	(NFR-03)	✓
Language model as detector, 9 scenarios	all 4 fires caught, 0 false alarms	(FR-08)	✓
Model answers never used when out of date	typical answer 7–21 s; simulation held to 1.4–3$\times$	(NFR-07)	✓

12. Dashboard

The dashboard is two browser tabs. They are meant to be read together, and each has its own job. The BuildSim viewer shows the building. It has heat maps of the true temperature, smoke and CO on the floors. It shows flames and smoke in the burning room, the sprinkler running, and every one of the 130 people walking. Alarming rooms are outlined in red, orange or yellow by state. The one escape route is a red line from the fire room, down the stairs, to a ground-floor exit. The control panel shows the system: what it is told, what it believes, and what it did about it.

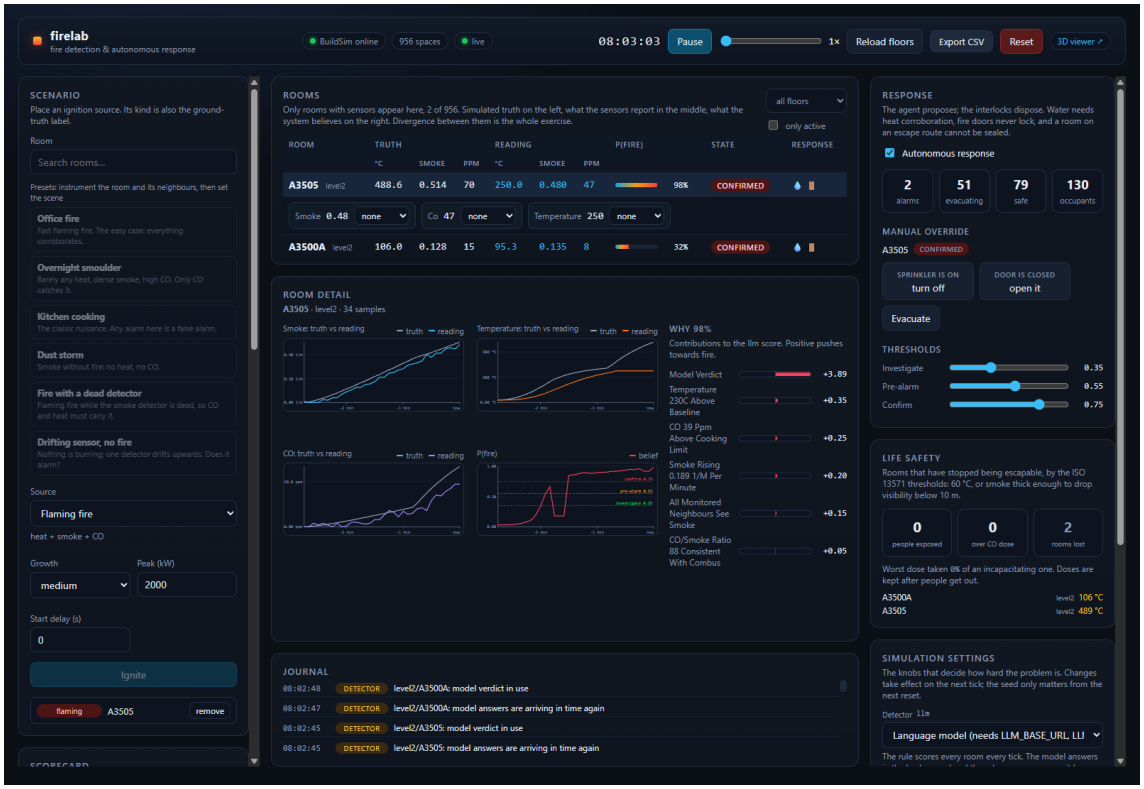


Figure 12: *The control panel during the office fire, with the language model as the detector. Rooms table: truth on the left, readings in the middle, P(fire) and state on the right. Room detail: truth against reading for each sensor type, and the "Why" bars. Here the bars show the model's own evidence, with its answer as the top bar. Journal: the model's hand-overs. Bottom right: the detector selector.*

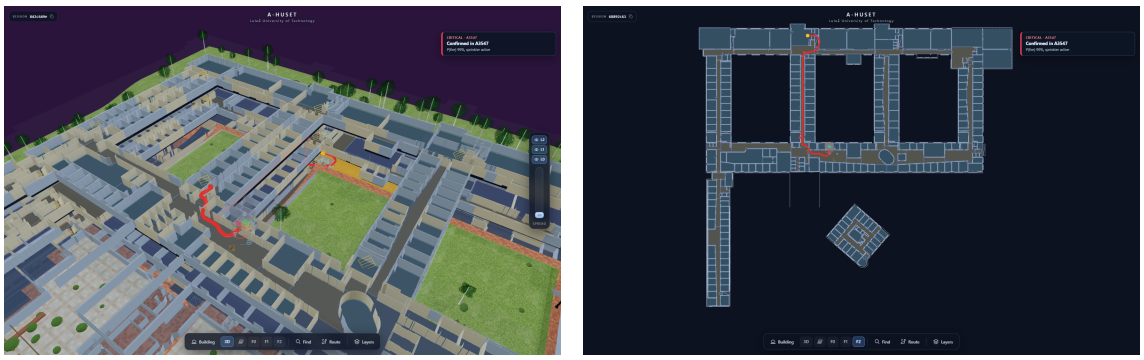


Figure 13: *The BuildSim viewer with firelab's overlays. Left: the 3D view, with the fire in the top-floor office and the escape route going down the stairwell. Right: the same route on the floor-2 plan, from the burning room to the exit.*

The most useful part of the panel is the least visual. The Rooms table puts the true value of each sensor type next to what its sensor reported. The room detail draws both on one chart. A maxed-out heat detector, a stuck smoke detector or a drifting one shows up as a gap between two lines. The operator of a real building never gets to see this, and it is what this project is about. Next to it, the "Why" bars split the probability into its parts. So 99 % is not a claim but a sum. With the model installed, the same bars show what the model said it weighed, with its answer as the top bar. The heading names which detector is speaking. The P(fire) trace in Figure 12 shows the model's first answer pulling the room back to normal before its second answer lifted it. A number on its own would hide that. The journal below records every state change and every command with its reason. It also records every refusal with the safety rule's reason, and every hand-over between the model and the rule. The scorecard grades the run as it happens: catches, misses, false alarms, the alarm delay and the RSET. You choose the detector in the settings panel, and you can change it mid-fire.

The speed dial then shows the cap the model sets. Everything on the panel arrives over the event stream, one snapshot per tick. The whole record can be saved as a labelled CSV.

13. Risks and critical reflection

The system does what the requirements ask. But the evaluation found five things that deserve more doubt than a tick mark gives them.

The dead-detector blind spot is real. A room with two of its three sensor types working stalled three thousandths below the confirm threshold. Its neighbour rescued it. That is a good outcome in this building, where offices share doorways with corridors. But it depends on the building's layout, not on the detector. The coverage penalty should be softened, or a confirmed neighbour should be allowed to confirm the room. Both are one-line changes. Choosing between them needs more than one run.

The margins are thinner than the zeros suggest. No false alarm went off, but cooking reached 0.749 against a threshold of 0.75. The waiting times are what really kept it quiet. A different kitchen, a bigger pan or a slightly different noise draw would push the probability over. The real question is whether it would then hold for 45 seconds. I did not measure that across enough seeds to say. The false-alarm rate here is zero in the runs I did, not zero in general.

Containment is modelled too kindly to the neighbours and too harshly to the fire room. Closing the doors halved the smoke next door, which is right. It also raised the fire room from 309 to 648 °C. That comes from the model: water only reduces the heat the fire releases, and a sealed room cannot run out of oxygen. In a zone model, the sprinkler would cool the hot gas layer and the fire would starve. The relative comparisons in Section 11 stand, but the absolute temperatures should not be quoted. The 35 °C bar in the sprinkler safety rule is also low, and the smoulder passed it in 18 s. A rate-of-rise condition, which checks how fast the temperature climbs, would be easier to defend.

One process is the right call for a simulator and the wrong one for a building. My proposal planned nine containers, and this report has two: the control panel and one backend service that holds all four zones. The reason, one simulated clock, is genuine, and the layering test keeps the zones separable. But nothing here shows fault isolation. A fire safety system that dies with its physics engine would not be certified. The way to split it is known and narrow. The sensing zone already sends readings per device. The actuation zone already takes commands with reasons. Those are the two topics a broker would carry, and simulated time would travel as a field in every message. The evacuation model is optimistic in the same direction. Getting 130 people out in 111 s is a walking-speed number. It has no queues at doors and no one who hesitates.

The language model is the slowest part of the system, and it added the least. In E7 it caught the same fires as the rule, one of them 30 s later, and it slowed the whole simulation down to a few times real time. It also runs on a big remote server, which is an odd fit for a project about intelligence at the edge, and its results change from run to run. The way I connected it is sound: it only sees sensor readings, its answer goes through the same waiting times and safety rules, and the rule covers any room it has not answered for. The weak part is the job I gave it. Scoring every room, again and again, is better done by a small trained model. The language model was most useful elsewhere: in the room with the dead smoke detector, where it trusted the other two sensors, and in the journal, where it explains itself in words. Next time I would only ask it when the rule is unsure or the sensors disagree. That keeps the useful part and drops most of the cost.

Risk / assumption	Impact	Mitigation / decision
Coverage penalty caps a room with two sensor types below confirm	A fire with a dead smoke detector is caught only through its neighbour	Soften the floor of the coverage factor, or let a confirmed neighbour confirm the room. Measure over many seeds
Cooking probability within 0.001 of the threshold	A false alarm under a different noise draw	Waiting times hold. Add a multi-seed false-alarm rate to the harness before tuning
Lumped physics with no oxygen limit and no direct cooling	Sealed fire rooms overheat. Absolute temperatures make the response look better or worse than it is	Compare results with each other, not as absolute values. A two-layer zone model is the next physics step
Heat check at 35 °C	Water released on a warm smoulder or a hot day	Require a rate of temperature rise as well as a level

Risk / assumption	Impact	Mitigation / decision
Single process, no fault isolation	A crash in one zone stops detection	Fine on the laptop. The zone seams become the broker topics when it is split
Evacuation without queues or hesitation	RSET is a lower bound	Report it as one. Add door capacity and a pre-movement delay per role
Hand-tuned detector weights	May not carry over to other buildings or sensor mixes	The CSV export and the Detector protocol are ready for a trained replacement. The rule is its baseline
Language model as the per-room detector	The simulation runs only 1.4–3× real time. One alarm came 30 s later. Results change between runs	The rule, the waiting times and the safety rules limit the harm. Only ask the model when the rule is unsure
Route drawn in one viewer tab, sent once	A second tab, or a reload at the wrong moment, sees no line	Re-sending on session change is done. A replay per tab on connect would finish it

If I had two more weeks, I would spend them on three things, in this order. First, a multi-seed false-alarm rate. Second, the coverage fix, with that rate as its test. Third, the trained detector my proposal promised. I would judge it against the rule on the same recorded runs, and on whether its “Why” can be drawn as bars. The system as it stands is small, explainable and measured. Its weak points are written down here rather than found during the demo.